

Question:

- Discuss the working principle of a direct-mapped cache. Include its addressing scheme (memory block addresses, tag, and valid bits) and explain how hits and misses are determined.

Answer:

- **Core Principle:** In a direct-mapped cache, every main memory block maps to exactly **one specific location** in the cache. The cache location is found using the formula:

$$\text{Cache Block Number} = (\text{Memory Block Address}) \pmod{\text{Total Cache Blocks}}$$

- **Addressing Scheme:** A memory address is divided into three fields:
 1. **Tag:** High-order bits used to identify if the cached block matches the requested memory block.
 2. **Index:** Middle bits used to select the specific row/slot in the cache.
 3. **Offset:** Low-order bits used to find the exact byte inside the cache block.
- **Valid Bit:** A single bit (1 or 0) stored in each cache row to indicate whether the data in that slot is valid/legal (1) or empty/garbage (0).
- **How Hits and Misses are Determined:**
 1. The CPU uses the **Index** bits to look up the specific row in the cache.
 2. It checks the **Valid Bit** of that row. If it is 0, it is a **Cache Miss**.
 3. If the valid bit is 1, the CPU compares its address **Tag** with the tag stored in that cache slot.
 - **Cache Hit:** If the tags match, data is delivered instantly.
 - **Cache Miss:** If the tags do not match, the requested block is brought from main memory, stored in this slot, and the tag/valid bits are updated.

Question 1 (from image_cd9b.png):

- A cache consists of 64 blocks, each having a block size of 16 bytes. Determine the cache block number to which the byte address 1200 is mapped.

Answer:

1. Find Memory Block Address:

$$\text{Memory Block Address} = \frac{\text{Byte Address}}{\text{Block Size}} = \frac{1200}{16} = 75$$

2. Find Cache Block Number:

$$\text{Cache Block Number} = (\text{Memory Block Address}) \pmod{\text{Total Cache Blocks}}$$

$$\text{Cache Block Number} = 75 \pmod{64} = 11$$

- Answer: Block 11

Question 2 (from image_cd9b.png):

- How many total bits are needed for a direct mapped cache with 64KB data in 1-word blocks? (Assuming a standard 32-bit architecture)

Answer:

1. Convert Sizes to Blocks:

- Block size = 1 word = 4 bytes = 2^2 bytes $\implies$ **2** bits for Offset.
- Total Cache Blocks = $\frac{64 \text{ KB}}{4 \text{ bytes}} = \frac{64 \times 1024}{4} = 16,384 = 2^{14}$ blocks $\implies$ **14** bits for Index.

2. Find Tag Bits:

- Tag bits = $32 - (\text{Index bits} + \text{Offset bits}) = 32 - (14 + 2) = \mathbf{16}$ bits.

3. Total Bits per Cache Row:

- Each entry needs: Valid bit(1) + Tag bits(16) + Data bits(32) = 49 bits.

4. Total Cache Size Required:

- Total bits = Total Cache Blocks $\times$ Bits per row = $16,384 \times 49 = \mathbf{802,816}$ bits (or ≈ 803 Kb).

- Slide er math gula dekhate bolse sir egula korar por ogula o dekhate hbe.

Question:

- Q5: Cache memory can be classified by how data maps from main memory. Explain the following three mapping architectures: a) Direct-Mapped Cache, b) Fully Associative Cache, c) Set-Associative Cache

Answer:

- **a) Direct-Mapped Cache:**
 - Each block of main memory maps to exactly **one specific slot** in the cache.
 - *Pros/Cons:* Simple and fast to look up, but causes frequent conflicts if multiple active blocks map to the same slot.
- **b) Fully Associative Cache:**
 - A block of main memory can be placed in **any available slot** anywhere in the cache.
 - *Pros/Cons:* Completely eliminates conflict misses, but requires expensive hardware because it must search all cache slots simultaneously.
- **c) Set-Associative Cache:**
 - The cache is divided into groups called **sets**, where each set contains a fixed number of slots (e.g., 2 or 4). A memory block maps to a specific set, but can take **any slot inside that set**.
 - *Pros/Cons:* A middle-ground compromise that provides the speed benefits of direct-mapped and the flexibility of fully associative.